TaskHub Kubernetes assignment

Submission Report

Yousef Elsayed Abdlebaset

Repository

github.com/yousefelsayed573/k8s-taskhub

Report guide

03 Cluster nodes and namespace

04 Deployments, StatefulSet, DaemonSet and CronJob

05 Frontend Pods and ReplicaSet

06 Frontend probes and resource settings

07 Backend probes and resource settings

08 StatefulSet, probes, resources and PVC mount

09 DaemonSet on the worker nodes

10 Networking, Ingress and PV/PVC inventory

11 Frontend and API reachability

12 Database persistence

13 emptyDir and hostPath demonstrations

14 NFS shared storage demonstration

15 Jobs: parallel workers and multiple completions

16 Jobs and CronJob execution

17 ReplicaSets, configuration and RBAC objects

18 RBAC authorization checks

19 Self-healing after frontend Pod deletion

20 Readiness failure state and recovery

Reading the screenshots: “visible” refers to the original screenshot. A later healthy state does not prove which repair command was used. Timestamps belong to different captures and should not be treated as a single simultaneous cluster snapshot.

Cluster nodes and namespace

Figure 01. Three Ready nodes - The control-plane node and two worker nodes are Ready. The output records Kubernetes v1.37.0, internal IP addresses and the container runtime.

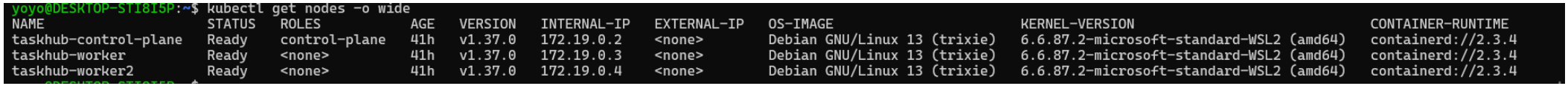
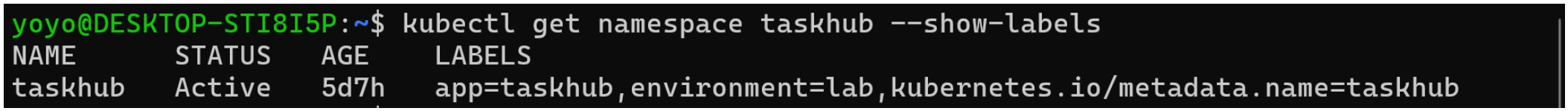


Figure 02. Active namespace and labels - The taskhub namespace is Active and carries app=taskhub and environment=lab labels.



Deployments, StatefulSet, DaemonSet and CronJob

Figure 03. Workload and scheduled-task status - Frontend and backend Deployments are 2/2 Ready; the database StatefulSet is 1/1. The node-agent DaemonSet has two desired/current/Ready instances. The maintenance CronJob has schedule */2 * * * *, SUSPEND=True and ACTIVE=0 in this final capture.

```
yoyo@DESKTOP-STI8I5P:~$ kubectl get deployments,statefulsets,daemonsets,cronjobs -n taskhub
NAME                                 READY   UP-TO-DATE   AVAILABLE   AGE
deployment.apps/taskhub-backend      2/2     2            2           5d3h
deployment.apps/taskhub-frontend     2/2     2            2           5d5h

NAME                                READY   AGE
statefulset.apps/taskhub-db         1/1     4d7h

NAME                                     DESIRED   CURRENT   READY   UP-TO-DATE   AVAILABLE   NODE SELECTOR   AGE
daemonset.apps/taskhub-node-agent        2         2         2       2            2           <none>          12h

NAME                                     SCHEDULE      TIMEZONE   SUSPEND   ACTIVE   LAST SCHEDULE   AGE
cronjob.batch/taskhub-maintenance        */2 * * * *   <none>     True      0        146m            168m
```

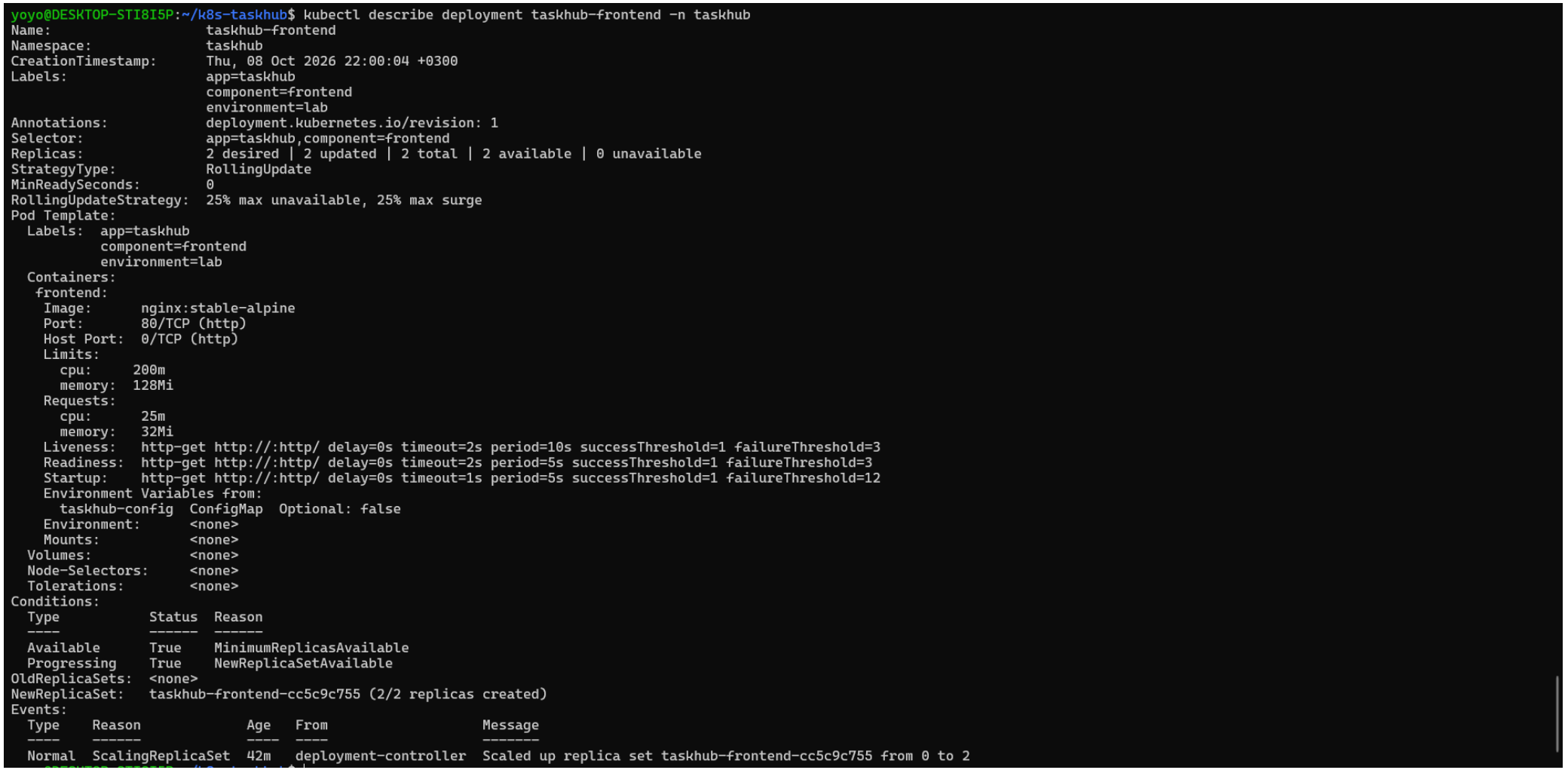
Frontend Pods and ReplicaSet

Figure 04. Frontend ReplicaSet and Pods - The frontend ReplicaSet has 2 desired, current and Ready replicas. Both frontend Pods are 1/1 Running, with one on each worker.

```
yoyo@DESKTOP-STI8I5P:~/k8s-taskhub$ kubectl get replicasets -n taskhub -l app=taskhub,component=frontend
NAME                        DESIRED   CURRENT   READY   AGE
taskhub-frontend-cc5c9c755   2         2         2       11m
yoyo@DESKTOP-STI8I5P:~/k8s-taskhub$ kubectl get pods -n taskhub -l app=taskhub,component=frontend -o wide
NAME                              READY   STATUS    RESTARTS   AGE   IP           NODE              NOMINATED NODE   READINESS GATES
taskhub-frontend-cc5c9c755-f4q8t   1/1     Running   0          11m   10.244.2.2   taskhub-worker2   <none>           <none>
taskhub-frontend-cc5c9c755-jsm5g   1/1     Running   0          11m   10.244.1.2   taskhub-worker    <none>           <none>
```

Frontend probes and resource settings

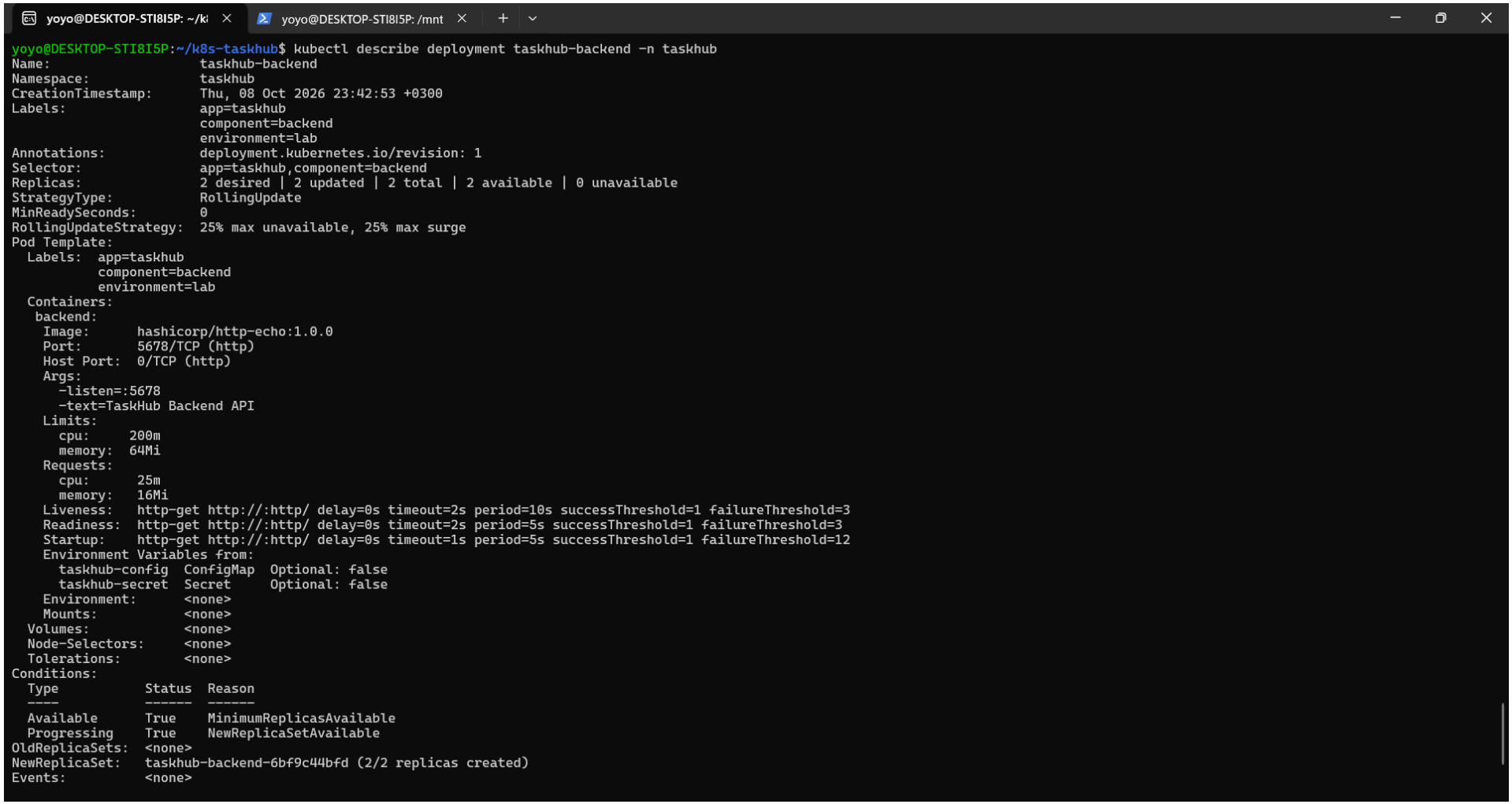
Figure 05. Frontend configuration - The frontend Deployment has two available replicas and uses nginx:stable-alpine. CPU/memory requests are 25m/32Mi; limits are 200m/128Mi. HTTP startup, readiness and liveness probes are visible.



```
yoyo@DESKTOP-STI8I5P:~/k8s-taskhub$ kubectl describe deployment taskhub-frontend -n taskhub
Name:                   taskhub-frontend
Namespace:              taskhub
CreationTimestamp:      Thu, 08 Oct 2026 22:00:04 +0300
Labels:                 app=taskhub
                        component=frontend
                        environment=lab
Annotations:            deployment.kubernetes.io/revision: 1
Selector:               app=taskhub,component=frontend
Replicas:               2 desired | 2 updated | 2 total | 2 available | 0 unavailable
StrategyType:           RollingUpdate
MinReadySeconds:        0
RollingUpdateStrategy:  25% max unavailable, 25% max surge
Pod Template:
  Labels:  app=taskhub
           component=frontend
           environment=lab
  Containers:
   frontend:
    Image:      nginx:stable-alpine
    Port:       80/TCP (http)
    Host Port:  0/TCP (http)
    Limits:
      cpu:     200m
      memory:  128Mi
    Requests:
      cpu:      25m
      memory:   32Mi
    Liveness:   http-get http://:http/ delay=0s timeout=2s period=10s successThreshold=1 failureThreshold=3
    Readiness:  http-get http://:http/ delay=0s timeout=2s period=5s successThreshold=1 failureThreshold=3
    Startup:    http-get http://:http/ delay=0s timeout=1s period=5s successThreshold=1 failureThreshold=12
    Environment Variables from:
      taskhub-config  ConfigMap  Optional: false
    Environment:      <none>
    Mounts:           <none>
  Volumes:            <none>
  Node-Selectors:     <none>
  Tolerations:        <none>
Conditions:
  Type           Status  Reason
  ----           ------  ------
  Available      True    MinimumReplicasAvailable
  Progressing    True    NewReplicaSetAvailable
OldReplicaSets:  <none>
NewReplicaSet:   taskhub-frontend-cc5c9c755 (2/2 replicas created)
Events:
  Type    Reason             Age   From                   Message
  ----    ------             ----  ----                   -------
  Normal  ScalingReplicaSet  42m   deployment-controller  Scaled up replica set taskhub-frontend-cc5c9c755 from 0 to 2
```

Backend probes and resource settings

Figure 06. Backend configuration - The backend Deployment has two available replicas and uses hashicorp/http-echo:1.0.0 on port 5678. Requests are 25m/16Mi; limits are 200m/64Mi. All three HTTP probes and ConfigMap/Secret references are visible.



StatefulSet, probes, resources and PVC mount

Figure 07. StatefulSet, probes and storage mount - The taskhub-db StatefulSet shows one Running Pod, postgres:16-alpine, a headless Service, and taskhub-db-pvc mounted at /var/lib/postgresql/data. Requests are 50m/128Mi; limits are 500m/384Mi. Startup, readiness and liveness use pg_isready.

```
yoyo@DESKTOP-STI8I5P:~$ kubectl describe statefulset taskhub-db -n taskhub
Name:                   taskhub-db
Namespace:              taskhub
CreationTimestamp:      Fri, 09 Oct 2026 20:32:26 +0300
Selector:               app=taskhub,component=database
Labels:                 app=taskhub
                        component=database
                        environment=lab
Annotations:            <none>
Service Name:           taskhub-db-headless
Pod Management Policy:  OrderedReady
Replicas:               1 desired | 1 total
Update Strategy:        RollingUpdate
  Partition:            0
  MaxUnavailable:       1
Persistent Volume Claim Retention Policy:
  WhenDeleted:  Retain
  WhenScaled:   Retain
Pods Status:    1 Running / 0 Waiting / 0 Succeeded / 0 Failed
Pod Template:
  Labels:  app=taskhub
           component=database
           environment=lab
  Containers:
   postgres:
    Image:      postgres:16-alpine
    Port:       5432/TCP (postgres)
    Host Port:  0/TCP (postgres)
    Limits:
      cpu:     500m
      memory:  384Mi
    Requests:
      cpu:      50m
      memory:   128Mi
    Liveness:   exec [sh -c pg_isready -U "$POSTGRES_USER" -d "$POSTGRES_DB"] delay=0s timeout=1s period=10s successThreshold=1 failureThreshold=3
    Readiness:  exec [sh -c pg_isready -U "$POSTGRES_USER" -d "$POSTGRES_DB"] delay=0s timeout=1s period=5s successThreshold=1 failureThreshold=3
    Startup:    exec [sh -c pg_isready -U "$POSTGRES_USER" -d "$POSTGRES_DB"] delay=0s timeout=1s period=5s successThreshold=1 failureThreshold=24
    Environment:
      POSTGRES_USER:      <set to the key 'DATABASE_USERNAME' in secret 'taskhub-secret'>  Optional: false
      POSTGRES_PASSWORD:  <set to the key 'DATABASE_PASSWORD' in secret 'taskhub-secret'>  Optional: false
      POSTGRES_DB:        taskhub
      PGDATA:             /var/lib/postgresql/data/pgdata
    Mounts:
      /var/lib/postgresql/data from db-data (rw)
  Volumes:
   db-data:
    Type:          PersistentVolumeClaim (a reference to a PersistentVolumeClaim in the same namespace)
    ClaimName:     taskhub-db-pvc
    ReadOnly:      false
  Node-Selectors:  <none>
  Tolerations:     <none>
Volume Claims:     <none>
Events:            <none>
yoyo@DESKTOP-STI8I5P:~$
```

DaemonSet on the worker nodes

Figure 08. Node-agent logs from both workers - Prefixed logs identify separate node-agent Pods reporting heartbeats on taskhub-worker and taskhub-worker2. The workload summary on page 4 records the DaemonSet count.

```
yoyo@DESKTOP-STI8I5P:~$ kubectl logs -n taskhub -l component=node-agent --prefix=true --tail=5
[pod/taskhub-node-agent-lgtv6/agent] TaskHub agent heartbeat on node: taskhub-worker2
[pod/taskhub-node-agent-lgtv6/agent] TaskHub agent heartbeat on node: taskhub-worker2
[pod/taskhub-node-agent-lgtv6/agent] TaskHub agent heartbeat on node: taskhub-worker2
[pod/taskhub-node-agent-lgtv6/agent] TaskHub agent heartbeat on node: taskhub-worker2
[pod/taskhub-node-agent-lgtv6/agent] TaskHub agent heartbeat on node: taskhub-worker2
[pod/taskhub-node-agent-r8ct9/agent] TaskHub agent heartbeat on node: taskhub-worker
[pod/taskhub-node-agent-r8ct9/agent] TaskHub agent heartbeat on node: taskhub-worker
[pod/taskhub-node-agent-r8ct9/agent] TaskHub agent heartbeat on node: taskhub-worker
[pod/taskhub-node-agent-r8ct9/agent] TaskHub agent heartbeat on node: taskhub-worker
[pod/taskhub-node-agent-r8ct9/agent] TaskHub agent heartbeat on node: taskhub-worker
```

Networking, Ingress and PV/PVC inventory

Figure 09. Services, Ingress and bound storage - Services include ClusterIP, a headless database Service, NodePort 30080, and a LoadBalancer with EXTERNAL-IP=<pending>. The Ingress class is traefik. Both 1Gi claims are Bound: database RWO/standard and NFS RWX. Their PV reclaim policies are Delete and Retain, respectively.



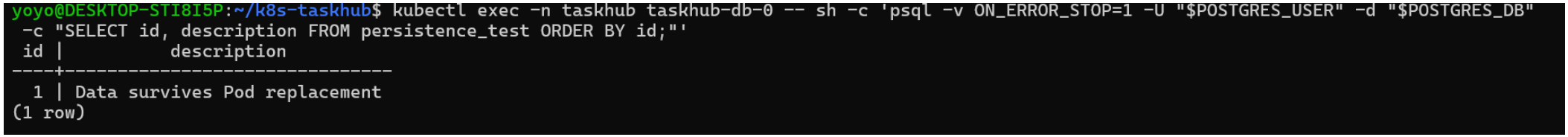
Frontend and API reachability

Figure 10. HTTP responses for frontend and API - Requests to http://127.0.0.1:8081/ and /api both return HTTP 200. The root returns the default nginx welcome HTML; /api returns TaskHub Backend API. This image does not show the Ingress routing configuration or controller status.



Database persistence

Figure 11. Database row is readable - A query in taskhub-db-0 returns id 1 and the description Data survives Pod replacement. The visible output proves the row is present; this image alone does not establish that the database Pod was deleted and recreated.



emptyDir and hostPath demonstrations

Figure 12. emptyDir demonstration after Pod recreation - The emptydir-demo Pod is deleted, recreated and becomes Ready. The subsequent check for /workspace/test.txt reports FILE NOT FOUND. The earlier creation of that file and the volume manifest are outside this capture.

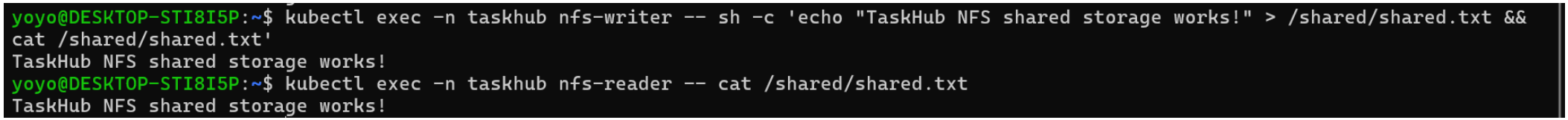
```
yoyo@DESKTOP-STI8I5P:~/k8s-taskhub$ kubectl delete pod emptydir-demo -n taskhub
pod "emptydir-demo" deleted from taskhub namespace
yoyo@DESKTOP-STI8I5P:~/k8s-taskhub$ kubectl apply -f ~/k8s-taskhub/storage/emptydir-demo.yaml
pod/emptydir-demo created
yoyo@DESKTOP-STI8I5P:~/k8s-taskhub$ kubectl wait --for=condition=Ready pod/emptydir-demo -n taskhub --timeout=120s
pod/emptydir-demo condition met
yoyo@DESKTOP-STI8I5P:~/k8s-taskhub$ kubectl exec -n taskhub emptydir-demo -- sh -c 'if [ -f /workspace/test.txt ]; then echo "FILE EXISTS"; else echo "FILE NOT FOUND"; fi
'
FILE NOT FOUND
```

Figure 13. hostPath demonstration and node placement - Reading /node-data/test.txt in hostpath-demo returns TaskHub hostPath persistent test. The Pod is 1/1 Running on taskhub-worker. A prior deletion/recreation and the hostPath manifest are not visible here.

```
yoyo@DESKTOP-STI8I5P:~/k8s-taskhub$ kubectl exec -n taskhub hostpath-demo -- sh -c 'if [ -f /node-data/test.txt ]; th
en cat /node-data/test.txt; else echo "FILE NOT FOUND"; fi'
TaskHub hostPath persistent test
yoyo@DESKTOP-STI8I5P:~/k8s-taskhub$ kubectl get pod hostpath-demo -n taskhub -o wide
NAME           READY   STATUS    RESTARTS   AGE    IP            NODE             NOMINATED NODE   READINESS GATES
hostpath-demo   1/1     Running   0          2m8s   10.244.1.10   taskhub-worker   <none>           <none>
```

NFS shared storage demonstration

Figure 14. Shared file read by two NFS demo Pods - nfs-writer writes /shared/shared.txt and nfs-reader reads the same text: TaskHub NFS shared storage works! This shows cross-Pod read/write behavior; placement on different nodes is not shown in this capture.



Jobs: parallel workers and multiple completions

Figure 15. Parallel-one Job and three worker logs - taskhub-job-parallel-one reports Complete, with COMPLETIONS displayed as 3/1 of 3. Three distinct worker Pods each log started and finished. The image does not show their start timestamps, overlap, or the Job successPolicy.

```
yoyo@DESKTOP-STI8I5P:~$ kubectl get job taskhub-job-parallel-one -n taskhub
NAME                       STATUS     COMPLETIONS   DURATION   AGE
taskhub-job-parallel-one   Complete   3/1 of 3      35s        29m
yoyo@DESKTOP-STI8I5P:~$ kubectl logs -n taskhub -l batch.kubernetes.io/job-name=taskhub-job-parallel-one --prefix=true --tail=5
[pod/taskhub-job-parallel-one-77jkd/worker] Worker taskhub-job-parallel-one-77jkd started
[pod/taskhub-job-parallel-one-77jkd/worker] Worker taskhub-job-parallel-one-77jkd finished
[pod/taskhub-job-parallel-one-f9l8b/worker] Worker taskhub-job-parallel-one-f9l8b started
[pod/taskhub-job-parallel-one-f9l8b/worker] Worker taskhub-job-parallel-one-f9l8b finished
[pod/taskhub-job-parallel-one-pcvql/worker] Worker taskhub-job-parallel-one-pcvql started
[pod/taskhub-job-parallel-one-pcvql/worker] Worker taskhub-job-parallel-one-pcvql finished
```

Figure 16. Multiple completions and observed concurrency - Two taskhub-job-multiple Pods are Running together. The Job watch progresses from 0/6 to 2/6, 4/6 and Complete 6/6, with a final duration of 77 seconds.

```
yoyo@DESKTOP-STI8I5P:~$ kubectl get pods -n taskhub -l batch.kubernetes.io/job-name=taskhub-job-multiple -o wide
NAME                         READY   STATUS    RESTARTS   AGE   IP            NODE             NOMINATED NODE   READINESS GATES
taskhub-job-multiple-kvv8x   1/1     Running   0          12s   10.244.1.12   taskhub-worker   <none>           <none>
taskhub-job-multiple-zfdwj   1/1     Running   0          12s   10.244.1.13   taskhub-worker   <none>           <none>
yoyo@DESKTOP-STI8I5P:~$ kubectl get job taskhub-job-multiple -n taskhub -w
NAME                   STATUS    COMPLETIONS   DURATION   AGE
taskhub-job-multiple   Running   0/6           20s        20s
taskhub-job-multiple   Running   0/6           25s        25s
taskhub-job-multiple   Running   0/6           26s        26s
taskhub-job-multiple   Running   2/6           26s        26s
taskhub-job-multiple   Running   2/6           29s        29s
taskhub-job-multiple   Running   2/6           30s        30s
taskhub-job-multiple   Running   2/6           51s        51s
taskhub-job-multiple   Running   2/6           52s        52s
taskhub-job-multiple   Running   4/6           52s        52s
taskhub-job-multiple   Running   4/6           54s        54s
taskhub-job-multiple   Running   4/6           76s        76s
taskhub-job-multiple   SuccessCriteriaMet   4/6           77s        77s
taskhub-job-multiple   Complete             6/6           77s        77s
```

Jobs and CronJob execution

Figure 17. Completed Jobs and maintenance executions - The single Job is Complete 1/1; the other two Job patterns are also Complete. Two maintenance Jobs and Pods have completed, with log execution times 21:58:03 and 22:00:02 UTC on 13 October. CronJob ownership is not displayed here; its schedule and later suspended state appear on page 4.

```
yoyo@DESKTOP-STI8I5P:~$ kubectl get jobs -n taskhub
NAME                          STATUS     COMPLETIONS   DURATION   AGE
taskhub-job-multiple          Complete   6/6           77s        78m
taskhub-job-parallel-one      Complete   3/1 of 3      35s        130m
taskhub-job-single            Complete   1/1           6s         3h12m
taskhub-maintenance-29865478  Complete   1/1           5s         2m18s
taskhub-maintenance-29865480  Complete   1/1           4s         18s
yoyo@DESKTOP-STI8I5P:~$ kubectl get pods -n taskhub -l component=maintenance
NAME                                READY   STATUS      RESTARTS   AGE
taskhub-maintenance-29865478-lm4nz  0/1     Completed   0          2m40s
taskhub-maintenance-29865480-s2wqc  0/1     Completed   0          40s
yoyo@DESKTOP-STI8I5P:~$ kubectl logs -n taskhub -l component=maintenance --prefix=true --tail=10
[pod/taskhub-maintenance-29865478-lm4nz/maintenance] TaskHub maintenance started
[pod/taskhub-maintenance-29865478-lm4nz/maintenance] Execution time: Tue Oct 13 21:58:03 UTC 2026
[pod/taskhub-maintenance-29865478-lm4nz/maintenance] TaskHub maintenance completed
[pod/taskhub-maintenance-29865480-s2wqc/maintenance] TaskHub maintenance started
[pod/taskhub-maintenance-29865480-s2wqc/maintenance] Execution time: Tue Oct 13 22:00:02 UTC 2026
[pod/taskhub-maintenance-29865480-s2wqc/maintenance] TaskHub maintenance completed
```

ReplicaSets, configuration and RBAC objects

Figure 18. ReplicaSets, RBAC objects and configuration - Backend and active frontend ReplicaSets each have two Ready replicas; the other frontend ReplicaSet is at zero. The taskhub-pod-reader ServiceAccount, Role and RoleBinding are listed. The ConfigMap has three data entries and the Opaque Secret has two; values are not displayed.

```
yoyo@DESKTOP-STI8I5P:~$ kubectl get replicasets -n taskhub -l app=taskhub
NAME                          DESIRED   CURRENT   READY   AGE
taskhub-backend-6bf9c44bfd    2         2         2       5d2h
taskhub-frontend-854bcb8dd7   0         0         0       32m
taskhub-frontend-cc5c9c755    2         2         2       5d4h
yoyo@DESKTOP-STI8I5P:~$ kubectl get serviceaccounts,roles,rolebindings -n taskhub
NAME                                 AGE
serviceaccount/default               5d6h
serviceaccount/taskhub-pod-reader    76m

NAME                                                  CREATED AT
role.rbac.authorization.k8s.io/taskhub-pod-reader-role   2026-10-13T22:32:42Z

NAME                                                         ROLE                              AGE
rolebinding.rbac.authorization.k8s.io/taskhub-pod-reader-binding   Role/taskhub-pod-reader-role   50m
yoyo@DESKTOP-STI8I5P:~$ kubectl get configmap/taskhub-config secret/taskhub-secret -n taskhub
NAME                         DATA   AGE
configmap/taskhub-config     3      5d5h

NAME                     TYPE     DATA   AGE
secret/taskhub-secret    Opaque   2      5d5h
```

RBAC authorization checks

Figure 19. Allowed and denied authorization checks - For system:serviceaccount:taskhub:taskhub-pod-reader, listing Pods in taskhub returns yes. Deleting Pods, reading Secrets, reading Pod logs and listing Pods in default each return no. These are kubectl auth can-i authorization results.



Self-healing after frontend Pod deletion

Figure 20. Frontend self-healing after deletion - The capture shows two Ready frontend Pods, deletion of the Pod ending f4q8t, a successful wait for two Ready replicas, and a new Pod ending 5bpkt at age 14s. Both final Pods are 1/1 Running. Two accidental shell command-not-found lines are retained unchanged.

```
yoyo@DESKTOP-STI8I5P:~$ kubectl get pods -n taskhub -l app=taskhub,component=frontend -o wide
NAME                              READY   STATUS    RESTARTS       AGE    IP           NODE              NOMINATED NODE   READINESS GATES
taskhub-frontend-cc5c9c755-f4q8t   1/1     Running   4 (2d8h ago)   5d4h   10.244.2.2   taskhub-worker2   <none>           <none>
taskhub-frontend-cc5c9c755-jsm5g   1/1     Running   3 (2d8h ago)   5d4h   10.244.1.3   taskhub-worker    <none>           <none>
yoyo@DESKTOP-STI8I5P:~$ taskhub-frontend-cc5c9c755-f4q8t
taskhub-frontend-cc5c9c755-jsm5g
taskhub-frontend-cc5c9c755-f4q8t: command not found
taskhub-frontend-cc5c9c755-jsm5g: command not found
yoyo@DESKTOP-STI8I5P:~$ kubectl delete pod taskhub-frontend-cc5c9c755-f4q8t -n taskhub
pod "taskhub-frontend-cc5c9c755-f4q8t" deleted from taskhub namespace
yoyo@DESKTOP-STI8I5P:~$ kubectl wait deployment/taskhub-frontend \
  -n taskhub \
  --for=jsonpath='{.status.readyReplicas}'=2 \
  --timeout=180s
deployment.apps/taskhub-frontend condition met
yoyo@DESKTOP-STI8I5P:~$ kubectl get pods -n taskhub -l app=taskhub,component=frontend -o wide
NAME                              READY   STATUS    RESTARTS       AGE    IP           NODE              NOMINATED NODE   READINESS GATES
taskhub-frontend-cc5c9c755-5bpkt   1/1     Running   0              14s    10.244.2.8   taskhub-worker2   <none>           <none>
taskhub-frontend-cc5c9c755-jsm5g   1/1     Running   3 (2d8h ago)   5d4h   10.244.1.3   taskhub-worker    <none>           <none>
yoyo@DESKTOP-STI8I5P:~$
```

Readiness failure state and recovery

Figure 21. Not-ready Pod during a frontend rollout - A new frontend Pod from ReplicaSet 854bcb8dd7 is Running but 0/1 Ready, while the two existing Pods remain 1/1. The Deployment reports READY=2/2, UP-TO-DATE=1 and AVAILABLE=2. Probe events and the failing probe configuration are not captured.

```
yoyo@DESKTOP-STI8I5P:~$ kubectl get pods -n taskhub \
  -l app=taskhub,component=frontend \
  -w
NAME                               READY   STATUS    RESTARTS       AGE
taskhub-frontend-854bcb8dd7-nfv7h   0/1     Running   0              14s
taskhub-frontend-cc5c9c755-f4q8t    1/1     Running   4 (2d8h ago)   5d3h
taskhub-frontend-cc5c9c755-jsm5g    1/1     Running   3 (2d8h ago)   5d3h
q^Cyoyo@DESKTOP-STI8I5P:kubectl get deployment taskhub-frontend -n taskhubhub
NAME              READY   UP-TO-DATE   AVAILABLE   AGE
taskhub-frontend   2/2     1            2           5d4h
```

Figure 22. Recovered frontend rollout - kubectl rollout status reports successfully rolled out. The Deployment is 2/2 Ready, two up-to-date and two available; both listed Pods are 1/1. No kubectl rollout undo command was captured, so the recovery mechanism is not asserted.

```
yoyo@DESKTOP-STI8I5P:~$ kubectl rollout status deployment/taskhub-frontend -n taskhub --timeout=180s

kubectl get deployment taskhub-frontend -n taskhub

kubectl get pods -n taskhub -l app=taskhub,component=frontend
deployment "taskhub-frontend" successfully rolled out
NAME              READY   UP-TO-DATE   AVAILABLE   AGE
taskhub-frontend   2/2     2            2           5d4h
NAME                               READY   STATUS    RESTARTS       AGE
taskhub-frontend-cc5c9c755-f4q8t   1/1     Running   4 (2d8h ago)   5d4h
taskhub-frontend-cc5c9c755-jsm5g   1/1     Running   3 (2d8h ago)   5d4h
```

Verification notes and source record

Original brief. The original assignment PDF must still be supplied to verify its exact wording, grading criteria and completeness. The inaccessible chat uploads may contain additional relevant screenshots.

Readiness test. The not-ready Pod and subsequent healthy rollout are visible. The failing probe, failure events and repair command are not shown. There is no screenshot of kubectl rollout undo.

Persistence. The database query shows a stored row. A before/deletion/replacement/after sequence is missing from the recovered images. hostPath output similarly does not include a full replacement sequence.

Networking and scheduled tasks. The LoadBalancer remains pending. HTTP responses and the Ingress object are captured separately. Maintenance executions are visible, but their owner references are not; the later CronJob inventory explicitly shows suspension.

Curation and integrity. 22 unique original PNGs are embedded at their source resolution with no resampling, cropping, reconstruction or text replacement. The duplicate 13 October 23:20:13 Job capture was omitted in favor of the clearer 23:37:41 capture.

Source and delivery. Sources: original files from the local Pictures/Screenshots folder. A separate source manifest records file names and SHA-256 hashes.